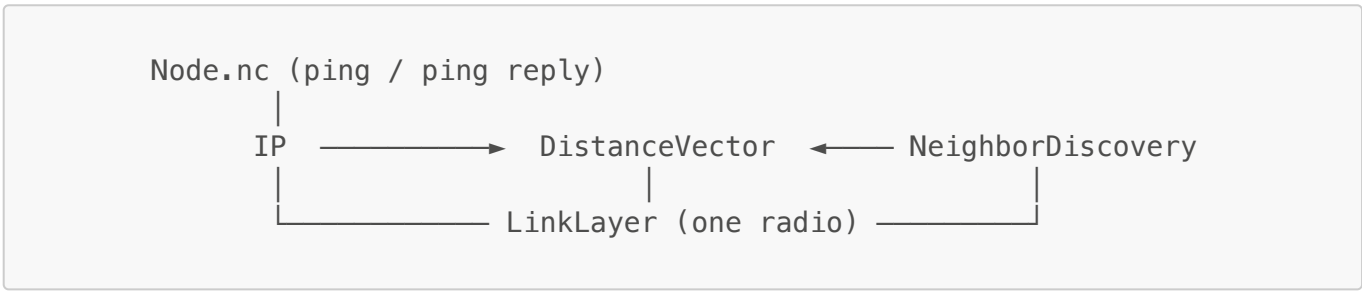
Project 2 Design: Distance-Vector Routing

Overview:

- Each node now has a routing table, so ping goes straight along one path instead of flooding the whole network.
- We split the work into two modules. DistanceVector keeps the routing table up to date, and IP uses the table to forward packets. IP never changes the table, it only asks DV "who is the next hop for this destination?"

Architecture:



- In Project 1 the link layer sorted packets by checking if dest was the broadcast address. That doesn't work anymore because DV packets are sent to one neighbor, so they would have ended up in flooding.
- To fix this we gave each type of traffic its own AM type:
  - AM_PACK (6) = neighbor discovery + flooding
  - AM_DV (11) = routing advertisements
  - AM_IP (12) = routed packets like ping
- Flooding is still there from Project 1, but ping doesn't use it anymore

Routing table:

- We used a plain array of 32 entries instead of the Hashmap. The Hashmap's get() gives back a copy, so every time we changed a cost we'd have to get it, change it, and insert it again. With an array we can just change the field.
- Each entry has: dest, nextHop, cost, age, and state (empty / valid / withdrawn)
- Slot 0 is always the node itself with cost 0, and it never gets replaced
- When neighbor discovery finds a new neighbor, we add a route to it with cost 1
- We only keep the best route for each destination, not every neighbor's whole table. If a route breaks, we find a new one from the next advertisement (about 6 seconds later).
- DV doesn't keep its own neighbor list. It asks neighbor discovery, and neighbor discovery tells DV when a neighbor is added or lost.

How routes get updated (Bellman-Ford):

When neighbor N says "I can reach D with cost c":

- our cost through N is c + 1, but if that's 16 or more it counts as unreachable

- we never change the route to ourselves
- we ignore advertisements from nodes that neighbor discovery hasn't confirmed yet
- if we don't have a route to D yet, we take it
- if N is already our next hop for D, we believe it even if the cost got worse. If N's path got longer, ours did too since we go through N.
- if N is a different neighbor, we only switch if it's strictly better
- if it's a tie, we keep the route we already have so the route doesn't keep flipping

Timers:

- Neighbor discovery: every 10s + random time. A neighbor is dropped after 4 rounds with no reply (~40s)
- Advertisements: every 5s + random time, so nodes don't all send at once
- Route timeout: if a route isn't refreshed for 6 rounds (~30s), it becomes unreachable
- Withdrawn routes: we keep advertising them as 16 for 6 more rounds so neighbors find out, then we delete them
- We picked 6 rounds so losing one or two advertisements doesn't break a route. Our loss test showed this.
- We didn't do triggered updates. It keeps things simple, but news only moves about one hop every 6 seconds.

Advertisements and poison reverse:

- Each neighbor gets its own copy of our table, sent only to them with TTL = 1. Advertisements are never forwarded.
- Poison reverse: if our route to D goes through neighbor N, we tell N our cost to D is 16. That way N never tries to reach D through us, which would make a loop.
- A packet only fits 9 entries (1 byte for the count + 2 bytes per entry = 19 of the 20 bytes), so bigger tables get split into multiple packets
- If a destination isn't in one packet, that doesn't mean it's gone. Routes only disappear if someone sends 16 or the route times out.

Forwarding (IP):

- The source sends with TTL = 15. Every node in the middle drops it if TTL <= 1, otherwise subtracts 1 and passes it on.
- If there's no route, we drop the packet and print why. We never fall back to flooding.

Tradeoffs:

- Only keeping the best route uses less memory, but when a link breaks we wait one round to find a new path
- No triggered updates is simpler, but slower to converge
- Poison reverse only stops loops between 2 nodes. Bigger loops are stopped by the max cost of 16 and the TTL.